# ECE551: Homework 2

Due: Monday, September 28th @ 11:55PM

For this homework, you *must*:

- Use informative module/signal/port names whenever at all reasonable to do so.
- Properly comment your design and testbench.
- Work individually.

## 1 (15pts) Saturate

See Exercise 05 specification. Did you complete it well? If not, polish it up.

**Submit: saturate.sv, saturate_tb.sv**, and console output from your self checks (i.e., your equivalent of a "yahoo tests passed").

## 2 (20pts) PID Math

See Exercise 06 specification. Did you complete it well? If not, polish it up.

**Submit: PID_Math.sv, PID_Math_tb.sv**, and waveform image closely mimicking the recommended test and results.

## 3 (15pts) Adder Exhaustive Test

Using **dataflow** RTL, implement a 4-bit wide adder that has the following interface:

| Signal | Direction | Description |
|---|---|---|
| A[3:0] | in | Operand 1 |
| B[3:0] | in | Operand 2 |
| cin | in | Carry in |
| Sum[3:0] | out | Sum of operand 1 and 2 |
| co | out | Carry out from addition of operands |

Create a testbench that **exhaustively** tests all combinations of inputs. The testbench should also instantiate or implement a behavioral implementation of the adder to be used as the "golden reference". The testbench should be self checking and should error out and stop if there is a miscompare between the "gold" model and your DUT. It should finish with a happy message if all goes well.

**Submit:**

- DUT: **adder.sv**
- testbench: **adder_tb.sv**
- image: proof that you ran it to completion successfully

# 4 (30pts) Segway Math

See Exercise 07 specification. Did you complete it well? If not, polish it up.

**Submit: SegwayMath.sv, SewayMath_tb.sv**, and waveforms (plural) showing the results of the 2 recommended simulations.

# 5 (20pts) Inferring FF

See Exercise 08 specification. Did you complete it well? If not, polish it up.

**Submit: HW2_prob5.sv**

- The comments should answer the questions about the latch posed above.
- The file should contain the model of a D-FF with an active high synchronous reset.
- The file should contain the model of a D-FF with asynchronous active low reset and an active high enable.
- The file should contain the model of a SR FF with active low asynchronous reset. SR meaning it has a S input that will set the flop, and a R input that will reset the flop, and it maintains state if neither S or R are high. It also has active low async reset. This is a handy style flop that we will use frequently.
- I would like you to use the always_ff construct of System Verilog. The file should contain (as comments) the answer to this question: Does the use of the always_ff construct ensure the logic will infer a flop? Looking for a little more than a simple yes/no answer.